

Project Design Phase
Solution Architecture

Date	13 July 2026
Team ID	SWTID-2026-9658
Project Name	Customer Registry
Maximum Marks	4 Marks

Solution Architecture:

1. Architectural Overview & Business Alignment

The solution architecture is designed to bridge the gap between untracked customer issues and active resolution by establishing a secure, scalable, and responsive MERN (MongoDB, Express, React, Node.js) Stack application. The architecture replaces manual tracking or "black box" email systems with a real-time, event-driven data flow.

2. Software Structure & Tiered Components

The Customer Care Registry is structured into three primary architectural layers:

A. Presentation Layer (Frontend - React.js)

- **Characteristics:** A single-page application (SPA) that acts as the user interface. It features a responsive layout designed for both desktop and mobile viewports.
- **Core Views:**
 - **Customer Portal:** Ticket submission forms with input validation and a personal status tracking history log.
 - **Agent Dashboard:** A centralized ticket queue with robust search, filter, and status modification controls.

B. Application & Logic Layer (Backend - Node.js & Express.js)

- **Characteristics:** A RESTful API backend that processes requests from the presentation layer and communicates with the database.
- **Core Controller Modules:**
 - **User Authentication Controller:** Manages secure user login, registration, and role-based access control (Customers vs. Support Agents).
 - **Ticket Controller:** Handles validation logic, creation, updates, and database retrieval queries for support complaints.

C. Data Layer (Database - MongoDB)

- **Characteristics:** A flexible, document-based non-relational database used to store application records.
- **Core Collections:**

- **Users Collection:** Stores credentials, roles, and profile details.
- **Tickets Collection:** Stores ticket IDs, customer IDs, detailed descriptions, attachments, status states (e.g., *Pending*, *In Progress*, *Resolved*), and update timestamps.

3. Data Flow and System Behavior

To define how the solution behaves, the system processes a customer complaint through the following structured data path:

Plaintext

[Customer UI] --(1. Submits Ticket Form via HTTPS POST)--> [Express Router]

|

(2. Server-side Validation)

|

[MongoDB Database] <--(4. Commits Document)-- [Ticket Controller / Mongoose]

|

(5. Returns Created Status)

|

[Customer UI] <--(6. Displays Real-time Success State & Ticket Log Updates)

- **Submission:** The user submits a structured complaint form via the React frontend.
- **Routing:** The presentation layer sends a secure HTTPS POST request containing the ticket payload to the Node/Express backend.
- **Authentication & Validation:** The backend validates the user's session token and verifies that the ticket fields meet database schema criteria.
- **Data Persistence:** The validated ticket data is sent to the MongoDB instance and written to the Tickets collection.
- **Confirmation:** Once committed, MongoDB sends a confirmation response back to the Express server, which responds with a 201 Created HTTP status code.
- **UI Refresh:** The React interface immediately updates the customer's dashboard feed with their new ticket details, eliminating user anxiety.

4. System Specifications & Non-Functional Requirements

- **Security:** JSON Web Tokens (JWT) are implemented in the Express backend to secure client-to-server communications and ensure role authorization.

- **Scalability:** The horizontally scalable nature of MongoDB's document architecture ensures the system can seamlessly handle spikes in concurrent ticket submissions.
- **Responsiveness:** The React client-side routing model provides near-instant page transitions and ticket search filtration without requiring full browser reloads.

Solution Architecture Diagram:

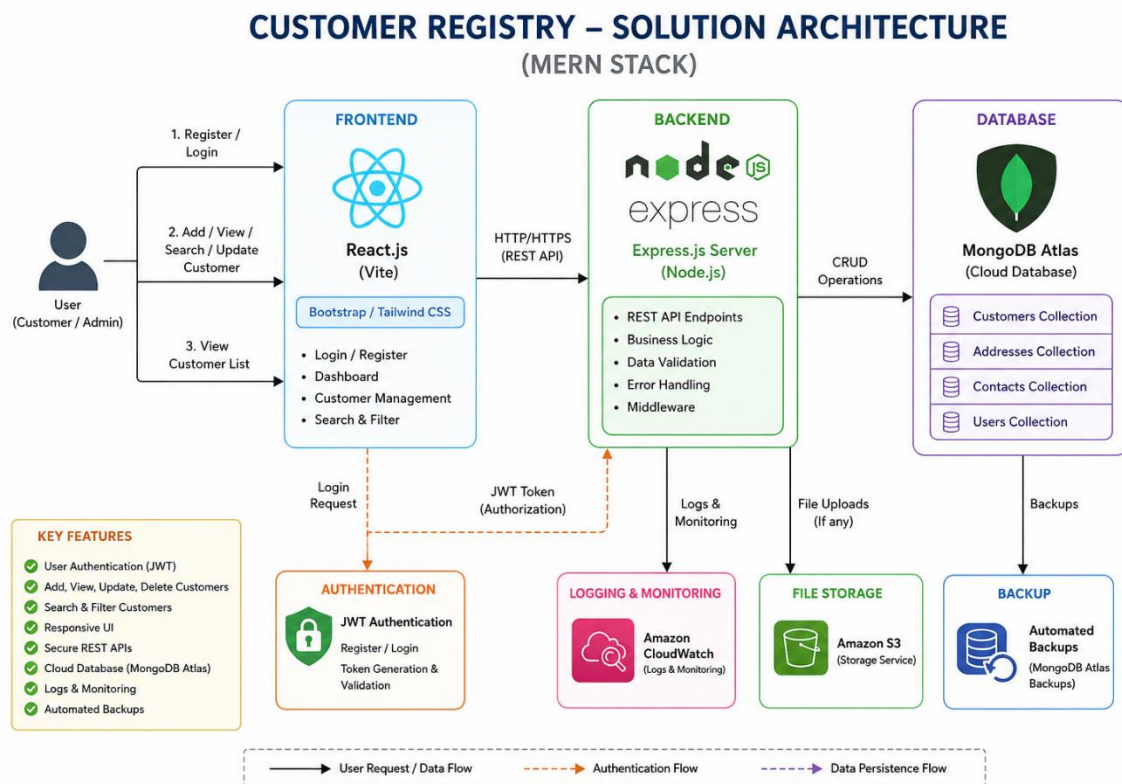


Figure 1: Architecture and data flow of the customer Registry